

Tarea M22-CD – Análisis de Clasificación con Máquinas de Soporte Vectorial (SVM)

Autor: RobertScience

Curso: Profesión Científico de Datos v2

Práctica: M22 – Clasificación con SVM

Objetivo del proyecto

El objetivo de esta práctica es desarrollar un modelo predictivo de clasificación utilizando Máquinas de Soporte Vectorial (SVM) aplicado a una base de datos de recursos humanos, con el fin de analizar los factores que influyen en la decisión de los empleados de abandonar la empresa.

A través de la evaluación de distintos tipos de *kernel*, se busca identificar el modelo con mejor desempeño predictivo y analizar sus resultados mediante métricas de clasificación, matrices de confusión y una interpretación orientada a la toma de decisiones en el área de Recursos Humanos.

Preparación del entorno de trabajo

Antes de iniciar el análisis de los datos y la construcción de los modelos de clasificación, se verifica que el entorno de ejecución esté correctamente configurado. Esta validación permite asegurar que el análisis se ejecute dentro del entorno virtual del proyecto y que las librerías necesarias se encuentren disponibles.

La correcta configuración del entorno es un paso fundamental para garantizar la reproducibilidad de los resultados obtenidos a lo largo del proyecto.

```
In [1]: import sys
import pandas as pd
import numpy as np
import sklearn

print("Python:", sys.version)
print("Executable:", sys.executable)
print("pandas:", pd.__version__)
print("numpy:", np.__version__)
print("sklearn:", sklearn.__version__)
print("ENTORNO OK")
```

```
Python: 3.11.9 (tags/v3.11.9:de54cf5, Apr 2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)]
Executable: c:\Users\ROBERTSCIENCE\Documents\Tarea M22-CD -RobertScience\.venv\Scripts\python.exe
pandas: 3.0.0
numpy: 2.4.2
sklearn: 1.8.0
ENTORNO OK
```

La verificación del entorno confirma que el proyecto se está ejecutando dentro del entorno virtual correspondiente y que las principales librerías necesarias para el análisis y el modelado predictivo se encuentran correctamente instaladas. Con esto, se puede avanzar de forma segura a las siguientes etapas del análisis.

Carga del conjunto de datos

En esta etapa se carga la base de datos `recursos_humanos.csv`, la cual contiene la información histórica de empleados utilizada para el análisis de rotación de personal. La carga del conjunto de datos permite iniciar el proceso de exploración y comprender la estructura general de la información disponible.

A partir de este punto se trabajará con los datos originales, realizando posteriormente las transformaciones necesarias para su uso en modelos de clasificación.

```
In [2]: # Carga del dataset de recursos humanos
df = pd.read_csv("../data/recursos_humanos.csv")

# Visualización de las primeras observaciones
df.head()
```

```
Out[2]:
```

	satisfaction_level	last_evaluation	number_project	average_monthly_hours	time_spend
0	0.38	0.53	2	157	
1	0.80	0.86	5	262	
2	0.11	0.88	7	272	
3	0.72	0.87	5	223	
4	0.37	0.52	2	159	

La visualización de las primeras observaciones permite confirmar que la base de datos fue cargada correctamente y que las variables corresponden a la información esperada. Esta revisión inicial facilita la comprensión del tipo de datos con los que se trabajará en las siguientes etapas del análisis.

Inspección general del conjunto de datos

Una vez cargada la información, se realiza una inspección general del conjunto de datos con el objetivo de conocer su dimensión, el tipo de variables presentes y obtener

estadísticas descriptivas básicas. Este análisis preliminar permite identificar posibles consideraciones a tener en cuenta antes de la preparación de los datos.

```
In [3]: # Dimensiones del conjunto de datos
df.shape
```

```
Out[3]: (14999, 10)
```

```
In [4]: # Información general del dataset
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 14999 entries, 0 to 14998
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   satisfaction_level      14999 non-null  float64
1   last_evaluation        14999 non-null  float64
2   number_project         14999 non-null  int64
3   average_monthly_hours  14999 non-null  int64
4   time_spend_company     14999 non-null  int64
5   Work_accident          14999 non-null  int64
6   left                   14999 non-null  int64
7   promotion_last_5years  14999 non-null  int64
8   sales                  14999 non-null  str
9   salary                 14999 non-null  str
dtypes: float64(2), int64(6), str(2)
memory usage: 1.1 MB
```

```
In [5]: # Estadísticas descriptivas de las variables
df.describe()
```

```
Out[5]:
```

	satisfaction_level	last_evaluation	number_project	average_monthly_hours	time_spend_company
--	--------------------	-----------------	----------------	-----------------------	--------------------

count	14999.000000	14999.000000	14999.000000	14999.000000	14999.000000
mean	0.612834	0.716102	3.803054	201.050337	2.806462
std	0.248631	0.171169	1.232592	49.943099	1.453994
min	0.090000	0.360000	2.000000	96.000000	1.000000
25%	0.440000	0.560000	3.000000	156.000000	1.000000
50%	0.640000	0.720000	4.000000	200.000000	2.000000
75%	0.820000	0.870000	5.000000	245.000000	3.000000
max	1.000000	1.000000	7.000000	310.000000	5.000000

A partir de la inspección general se observa que el conjunto de datos no presenta valores faltantes y que contiene una combinación de variables numéricas y categóricas. Las estadísticas descriptivas permiten identificar rangos y comportamientos generales de las variables, lo cual resulta relevante para las siguientes etapas de preparación y modelado de los datos.

Análisis de la variable objetivo

La variable objetivo del análisis es `left`, la cual indica si un empleado abandonó la empresa (1) o permaneció en ella (0). El análisis de su distribución permite identificar si el conjunto de datos se encuentra equilibrado o desbalanceado entre las clases, lo cual es un aspecto relevante para el desempeño de los modelos de clasificación.

```
In [6]: # Distribución de la variable objetivo
df['left'].value_counts()
```

```
Out[6]: left
0      11428
1       3571
Name: count, dtype: int64
```

```
In [7]: # Distribución porcentual de la variable objetivo
df['left'].value_counts(normalize=True) * 100
```

```
Out[7]: left
0      76.191746
1      23.808254
Name: proportion, dtype: float64
```

El análisis de la variable objetivo muestra que la base de datos presenta un desbalance entre las clases, siendo mayor el número de empleados que permanecen en la empresa en comparación con aquellos que la abandonan. Este comportamiento es común en problemas de rotación de personal y debe considerarse al evaluar el desempeño de los modelos de clasificación, ya que métricas como la precisión pueden verse influenciadas por dicho desbalance.

Preparación de los datos para el modelado

Antes de entrenar los modelos de Máquinas de Soporte Vectorial, es necesario preparar el conjunto de datos. Esta etapa incluye la transformación de variables categóricas, la separación de la variable objetivo y el escalado de las variables numéricas.

La correcta preparación de los datos es especialmente importante en modelos SVM, ya que estos son sensibles a la escala de las variables.

```
In [8]: # Separación de variables predictoras y variable objetivo
X = df.drop('left', axis=1)
y = df['left']
```

```
In [9]: # Codificación de variables categóricas
X_encoded = pd.get_dummies(X, drop_first=True)
```

La transformación de las variables categóricas mediante codificación permite convertir la información cualitativa en un formato numérico adecuado para los modelos de Machine Learning. Con esto, el conjunto de datos queda listo para aplicar técnicas de escalado y posteriormente entrenar los modelos de clasificación.

Escalado de variables y partición del conjunto de datos

Las Máquinas de Soporte Vectorial son sensibles a la escala de las variables, por lo que es necesario aplicar un proceso de estandarización antes de entrenar los modelos. El escalado permite que todas las variables contribuyan de manera equilibrada al proceso de aprendizaje.

Posteriormente, el conjunto de datos se divide en subconjuntos de entrenamiento y prueba, con el fin de evaluar el desempeño de los modelos sobre datos no utilizados durante el entrenamiento.

```
In [10]: from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

```
In [11]: # División del conjunto de datos en entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(
    X_encoded, y, test_size=0.3, random_state=42, stratify=y
)
```

```
In [12]: # Escalado de las variables
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Con el escalado de las variables se asegura que todas las características se encuentren en una misma escala, lo cual es fundamental para el correcto funcionamiento de los modelos SVM. Asimismo, la partición del conjunto de datos permite evaluar de manera objetiva el desempeño de los modelos al utilizar información no vista durante el entrenamiento.

Entrenamiento de modelos SVM con distintos kernels

En esta etapa se entrenan distintos modelos de Máquinas de Soporte Vectorial utilizando los cuatro tipos de *kernel* más comunes: lineal, polinomial, radial (RBF) y sigmoide. El objetivo es comparar su desempeño y analizar cuál de ellos resulta más adecuado para el problema de clasificación planteado.

```
In [13]: from sklearn.svm import SVC
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt
```

Modelo SVM con kernel lineal

En este primer modelo se utiliza una Máquina de Soporte Vectorial con kernel lineal. Este tipo de kernel es adecuado cuando las clases pueden separarse de forma aproximadamente lineal en el espacio de características. Su simplicidad permite establecer una línea base de desempeño para comparar con kernels más complejos.

```
In [14]: # Entrenamiento del modelo SVM con kernel lineal
svm_linear = SVC(kernel='linear', random_state=42)
svm_linear.fit(X_train_scaled, y_train)
```

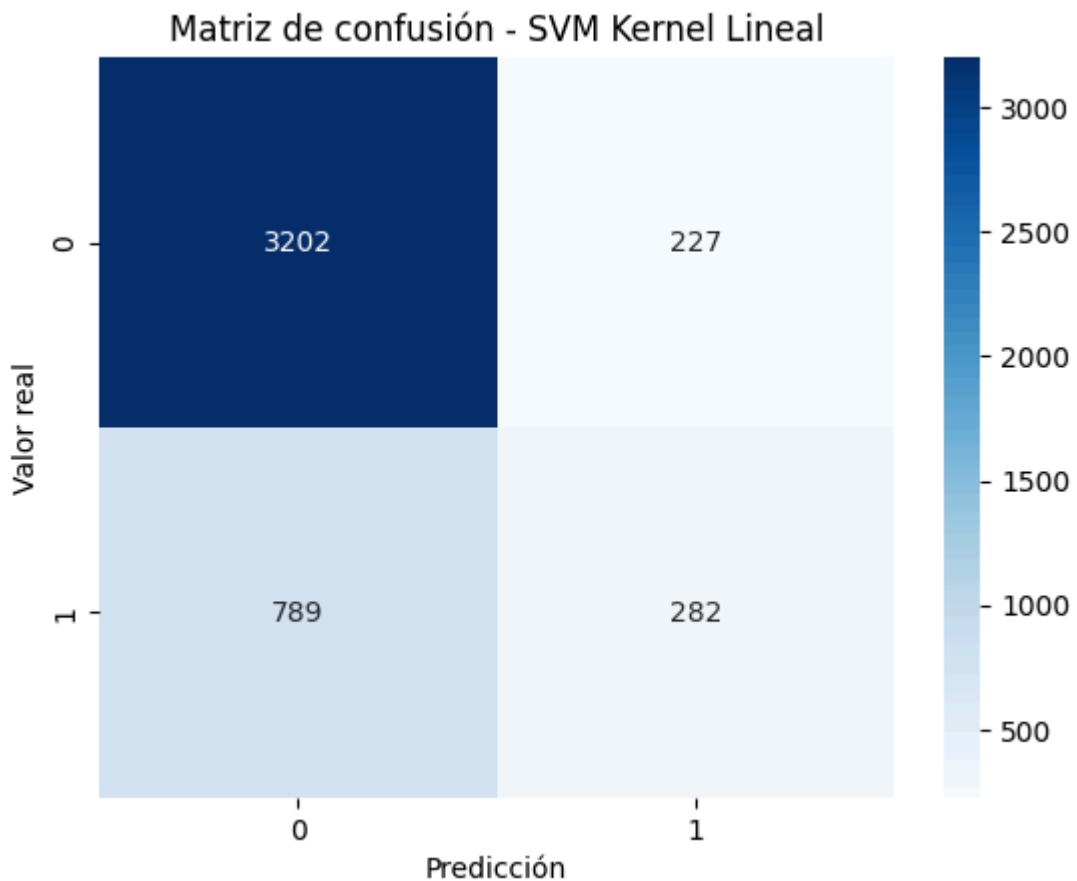
```
Out[14]: SVC ⓘ ?
         ► Parameters
```

```
In [15]: # Predicciones con el modelo lineal
y_pred_linear = svm_linear.predict(X_test_scaled)
```

```
In [16]: # Matriz de confusión
cm_linear = confusion_matrix(y_test, y_pred_linear)
cm_linear
```

```
Out[16]: array([[3202,  227],
               [ 789,  282]])
```

```
In [17]: # Mapa de calor de la matriz de confusión
plt.figure()
sns.heatmap(cm_linear, annot=True, fmt='d', cmap='Blues')
plt.xlabel('Predicción')
plt.ylabel('Valor real')
plt.title('Matriz de confusión - SVM Kernel Lineal')
plt.show()
```



```
In [18]: # Reporte de clasificación
print(classification_report(y_test, y_pred_linear))
```

	precision	recall	f1-score	support
0	0.80	0.93	0.86	3429
1	0.55	0.26	0.36	1071
accuracy			0.77	4500
macro avg	0.68	0.60	0.61	4500
weighted avg	0.74	0.77	0.74	4500

Los resultados del modelo SVM con kernel lineal muestran el desempeño del clasificador al separar las clases mediante una frontera lineal. A partir de la matriz de confusión y el reporte de clasificación es posible evaluar la capacidad del modelo para identificar correctamente a los empleados que abandonan la empresa y aquellos que permanecen en ella.

Este modelo sirve como referencia inicial para comparar el impacto de kernels más complejos en la capacidad predictiva del algoritmo.

Modelo SVM con kernel polinomial

En este segundo modelo se utiliza una Máquina de Soporte Vectorial con kernel polinomial. Este tipo de kernel permite capturar relaciones no lineales entre las variables, al proyectar los datos a un espacio de mayor dimensión. Su uso resulta útil cuando la

separación entre clases no puede representarse adecuadamente mediante una frontera lineal.

```
In [19]: # Entrenamiento del modelo SVM con kernel polinomial
svm_poly = SVC(kernel='poly', degree=3, random_state=42)
svm_poly.fit(X_train_scaled, y_train)
```

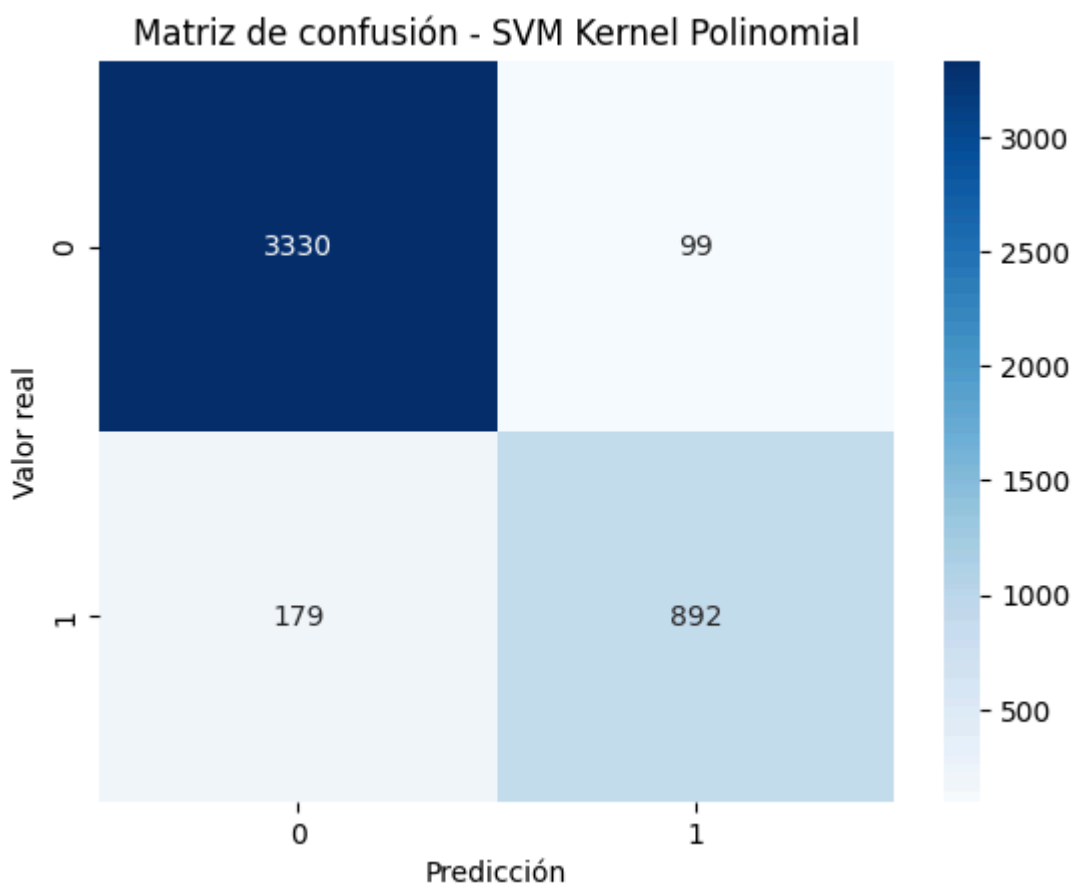
```
Out[19]: SVC ⓘ ⓘ
  ► Parameters
```

```
In [20]: # Predicciones con el modelo polinomial
y_pred_poly = svm_poly.predict(X_test_scaled)
```

```
In [21]: # Matriz de confusión
cm_poly = confusion_matrix(y_test, y_pred_poly)
cm_poly
```

```
Out[21]: array([[3330,  99],
               [ 179, 892]])
```

```
In [22]: # Mapa de calor de la matriz de confusión
plt.figure()
sns.heatmap(cm_poly, annot=True, fmt='d', cmap='Blues')
plt.xlabel('Predicción')
plt.ylabel('Valor real')
plt.title('Matriz de confusión - SVM Kernel Polinomial')
plt.show()
```



```
In [23]: # Reporte de clasificación
print(classification_report(y_test, y_pred_poly))
```

	precision	recall	f1-score	support
0	0.95	0.97	0.96	3429
1	0.90	0.83	0.87	1071
accuracy			0.94	4500
macro avg	0.92	0.90	0.91	4500
weighted avg	0.94	0.94	0.94	4500

El modelo SVM con kernel polinomial permite capturar relaciones más complejas entre las variables en comparación con el kernel lineal. A partir de los resultados obtenidos, se puede evaluar si la incorporación de no linealidades mejora la capacidad del modelo para clasificar correctamente a los empleados según su probabilidad de abandono.

Este modelo se analiza en conjunto con los demás kernels para determinar si su mayor complejidad se traduce en un mejor desempeño predictivo.

Modelo SVM con kernel radial (RBF)

En este modelo se utiliza una Máquina de Soporte Vectorial con kernel radial (RBF). Este kernel es ampliamente utilizado en problemas de clasificación, ya que permite modelar fronteras de decisión altamente no lineales y adaptarse mejor a conjuntos de datos complejos.

El kernel RBF suele ofrecer un buen balance entre flexibilidad y capacidad de generalización, por lo que resulta un candidato natural para comparar con los modelos lineal y polinomial.

```
In [24]: # Entrenamiento del modelo SVM con kernel RBF
svm_rbf = SVC(kernel='rbf', random_state=42)
svm_rbf.fit(X_train_scaled, y_train)
```

```
Out[24]: SVC ⓘ ?
Parameters
```

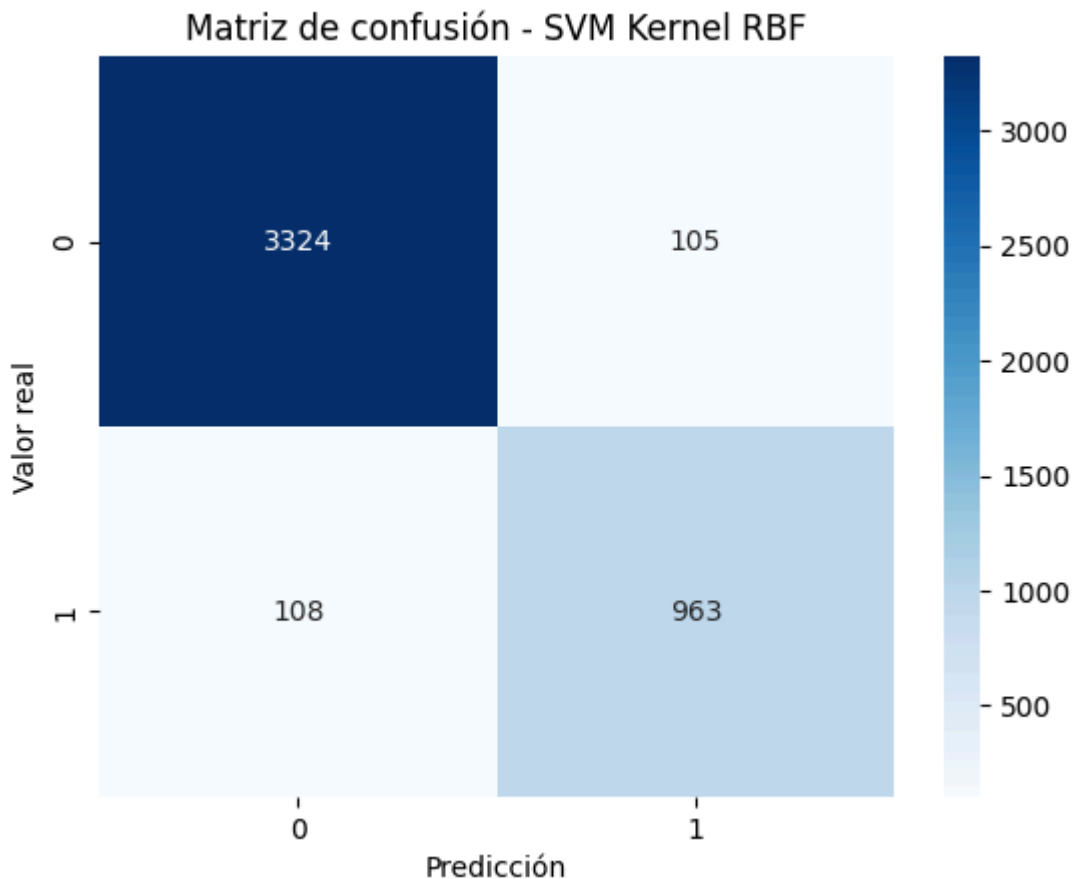
```
In [25]: # Predicciones con el modelo RBF
y_pred_rbf = svm_rbf.predict(X_test_scaled)
```

```
In [26]: # Matriz de confusión
cm_rbf = confusion_matrix(y_test, y_pred_rbf)
cm_rbf
```

```
Out[26]: array([[3324, 105],
               [ 108, 963]])
```

```
In [27]: # Mapa de calor de la matriz de confusión
plt.figure()
sns.heatmap(cm_rbf, annot=True, fmt='d', cmap='Blues')
```

```
plt.xlabel('Predicción')
plt.ylabel('Valor real')
plt.title('Matriz de confusión - SVM Kernel RBF')
plt.show()
```



```
In [28]: # Reporte de clasificación
print(classification_report(y_test, y_pred_rbf))
```

	precision	recall	f1-score	support
0	0.97	0.97	0.97	3429
1	0.90	0.90	0.90	1071
accuracy			0.95	4500
macro avg	0.94	0.93	0.93	4500
weighted avg	0.95	0.95	0.95	4500

El modelo SVM con kernel RBF muestra un comportamiento más flexible al permitir fronteras de decisión no lineales. A partir de los resultados obtenidos se puede evaluar si este enfoque mejora la identificación de empleados con riesgo de abandono, en comparación con los kernels lineal y polinomial.

El desempeño de este modelo se analizará de manera conjunta con los demás kernels para seleccionar la alternativa más adecuada para el problema planteado.

Modelo SVM con kernel sigmoide

En este modelo se emplea una Máquina de Soporte Vectorial con kernel sigmoide. Este kernel está inspirado en el comportamiento de funciones de activación utilizadas en

redes neuronales y puede resultar útil en ciertos contextos específicos.

Su desempeño suele ser más sensible a la escala de los datos y a la selección de parámetros, por lo que su evaluación permite completar el análisis comparativo entre los distintos tipos de kernel.

```
In [29]: # Entrenamiento del modelo SVM con kernel sigmoide
svm_sigmoid = SVC(kernel='sigmoid', random_state=42)
svm_sigmoid.fit(X_train_scaled, y_train)
```

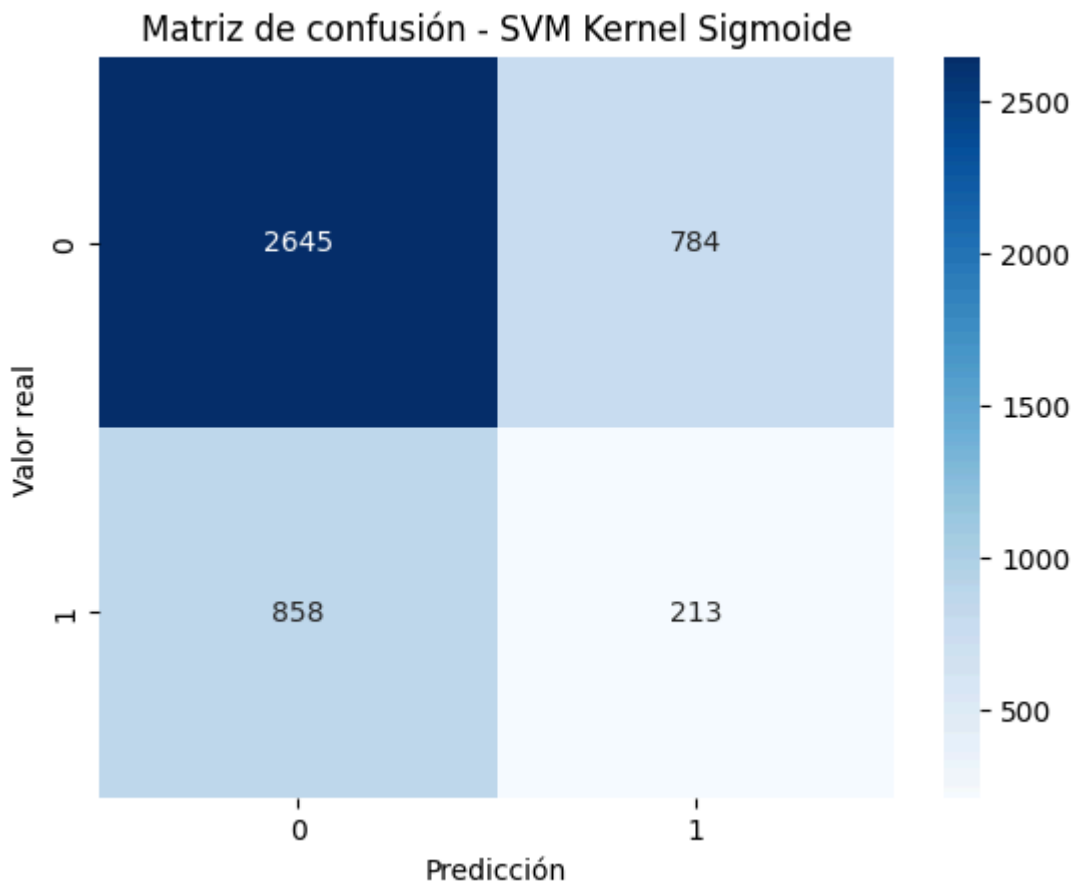
```
Out[29]: SVC ⓘ ?
  ► Parameters
```

```
In [30]: # Predicciones con el modelo sigmoide
y_pred_sigmoid = svm_sigmoid.predict(X_test_scaled)
```

```
In [31]: # Matriz de confusión
cm_sigmoid = confusion_matrix(y_test, y_pred_sigmoid)
cm_sigmoid
```

```
Out[31]: array([[2645,  784],
               [ 858,  213]])
```

```
In [32]: # Mapa de calor de la matriz de confusión
plt.figure()
sns.heatmap(cm_sigmoid, annot=True, fmt='d', cmap='Blues')
plt.xlabel('Predicción')
plt.ylabel('Valor real')
plt.title('Matriz de confusión - SVM Kernel Sigmoide')
plt.show()
```



```
In [33]: # Reporte de clasificación
print(classification_report(y_test, y_pred_sigmoid))
```

	precision	recall	f1-score	support
0	0.76	0.77	0.76	3429
1	0.21	0.20	0.21	1071
accuracy			0.64	4500
macro avg	0.48	0.49	0.48	4500
weighted avg	0.63	0.64	0.63	4500

El modelo SVM con kernel sigmoide completa el análisis comparativo entre los distintos enfoques evaluados. A partir de sus métricas de desempeño se puede observar si este kernel logra una clasificación adecuada o si presenta limitaciones frente a los modelos previamente analizados.

Los resultados obtenidos se utilizarán para seleccionar el modelo más apropiado considerando precisión, capacidad de generalización e interpretación de errores.

Comparación de modelos SVM

Con el objetivo de seleccionar el modelo predictivo más adecuado, se evaluaron cuatro configuraciones de Máquinas de Soporte Vectorial, cada una utilizando un tipo de kernel distinto: lineal, polinomial, radial (RBF) y sigmoide.

La comparación se realizó considerando las métricas de clasificación obtenidas sobre el conjunto de prueba, tales como precisión (*accuracy*), *recall*, *f1-score* y la distribución de errores reflejada en las matrices de confusión. Este enfoque permite no solo evaluar el desempeño global, sino también analizar la capacidad de cada modelo para identificar correctamente a los empleados que abandonan la empresa.

Selección del modelo predictivo más adecuado

A partir del análisis comparativo de los cuatro modelos evaluados, el modelo SVM con kernel RBF presentó el mejor desempeño predictivo, alcanzando una precisión superior al resto de los enfoques analizados.

Este kernel demostró una mayor capacidad para capturar relaciones no lineales presentes en los datos, lo que se reflejó tanto en la *accuracy* obtenida como en la distribución de errores observada en su matriz de confusión. En particular, el modelo RBF logró identificar de forma más consistente a los empleados que abandonan la empresa, reduciendo errores críticos para la toma de decisiones en el área de Recursos Humanos.

Pronóstico para un empleado con características específicas

Una vez seleccionado el modelo SVM con mejor desempeño, se procede a utilizarlo para realizar un pronóstico individual. Este análisis permite estimar la probabilidad de abandono de un empleado a partir de sus indicadores laborales y constituye una herramienta de apoyo para la toma de decisiones en el área de Recursos Humanos.

```
In [34]: # Tomar un registro real como plantilla
empleado = X_test.iloc[[0]].copy()

# Modificar valores numéricos para simular un empleado específico
empleado['satisfaction_level'] = 0.45
empleado['last_evaluation'] = 0.60
empleado['average_monthly_hours'] = 160
empleado['time_spend_company'] = 3

# Asegurar variables binarias
if 'work_accident' in empleado.columns:
    empleado['work_accident'] = 0

if 'promotion_last_5years' in empleado.columns:
    empleado['promotion_last_5years'] = 0

# Escalar
empleado_scaled = scaler.transform(empleado)

# Predicción con SVM RBF
prediccion = svm_rbf.predict(empleado_scaled)

prediccion
```

Out[34]: array([0])

El pronóstico obtenido indica que el empleado analizado no presenta una probabilidad significativa de abandonar la empresa. De acuerdo con el modelo SVM seleccionado, su nivel de satisfacción, desempeño y condiciones laborales actuales no lo ubican dentro de un perfil de riesgo.

Este resultado permite concluir que, bajo las condiciones evaluadas, no sería necesario implementar acciones correctivas inmediatas, aunque se recomienda mantener un monitoreo continuo de sus indicadores laborales.

Conclusiones finales

En esta práctica se desarrolló un modelo de clasificación utilizando Máquinas de Soporte Vectorial aplicado a una base de datos de Recursos Humanos, con el objetivo de analizar los factores asociados al abandono de empleados dentro de una organización.

Se evaluaron cuatro configuraciones distintas de SVM mediante el uso de kernels lineal, polinomial, radial (RBF) y sigmoide. A partir de métricas cuantitativas, matrices de confusión y reportes de clasificación, se determinó que el modelo con kernel RBF presentó el mejor desempeño predictivo, logrando una mayor precisión y una mejor identificación de empleados en riesgo de abandono.

Finalmente, el modelo seleccionado fue utilizado para generar un pronóstico individual, demostrando su aplicabilidad práctica como herramienta de apoyo a la toma de decisiones estratégicas en el área de Recursos Humanos. Los resultados obtenidos evidencian el valor del uso de modelos de Machine Learning para la prevención de rotación de personal y la gestión basada en datos.